AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Wydział Elektrotechniki, Automatyki, Informatyki i Inżynierii Biomedycznej

Projekt

Chirurg AR–Voice Control

Z przedmiotu:

Zastosowanie wirtualnej rzeczywistości w inżynierii biomedycznej

Autorzy: *Mateusz Piotrowski*
Weronika Solarska
Jakub Kalina
Ewa Rychlicka
Magdalena Suchan

Kierunek studiów: Inżynieria Biomedyczna

Kraków, 2026.

1. Wprowadzenie 3
2. Cel i zakres projektu 3
3. Architektura systemu 3
4. Moduł klienta głosowego 4
5. Moduł Slicer 3D 6
6. Komunikacja i protokół wymiany danych 7
7. Funkcjonalność systemu 7
8. Specyfikacja interfejsu głosowego 8
8.1. Klasyfikacja komend nawigacyjnych (Widoki 2D) 8
8.2. Zarządzanie strukturami anatomicznymi (Landmarks) 8
9. Instalacja i konfiguracja 9
9.1. Przygotowanie Klienta Głosowego 9
9.2. Integracja z oprogramowaniem 3D Slicer 9
9.3. Procedura startowa systemu 9
9.4. Zakończenie pracy systemu 9
10. Testy funkcjonalne 9
11. Ograniczenia systemu 10
12. Podsumowanie 10

1. Wprowadzenie

Moduł ChirurgAR – Voice Control jest rozszerzeniem systemu wizualizacji medycznej, które umożliwia sterowanie środowiskiem 3D Slicer za pomocą komend głosowych w języku polskim. System wykorzystuje rozpoznawanie mowy offline oparte o Vosk, co pozwala na działanie bez połączenia z Internetem i zwiększa bezpieczeństwo danych medycznych. Celem projektu jest poprawa ergonomii pracy użytkownika (np. lekarza), poprzez eliminację konieczności korzystania z myszy i klawiatury podczas analizy danych obrazowych.

2. Cel i zakres projektu

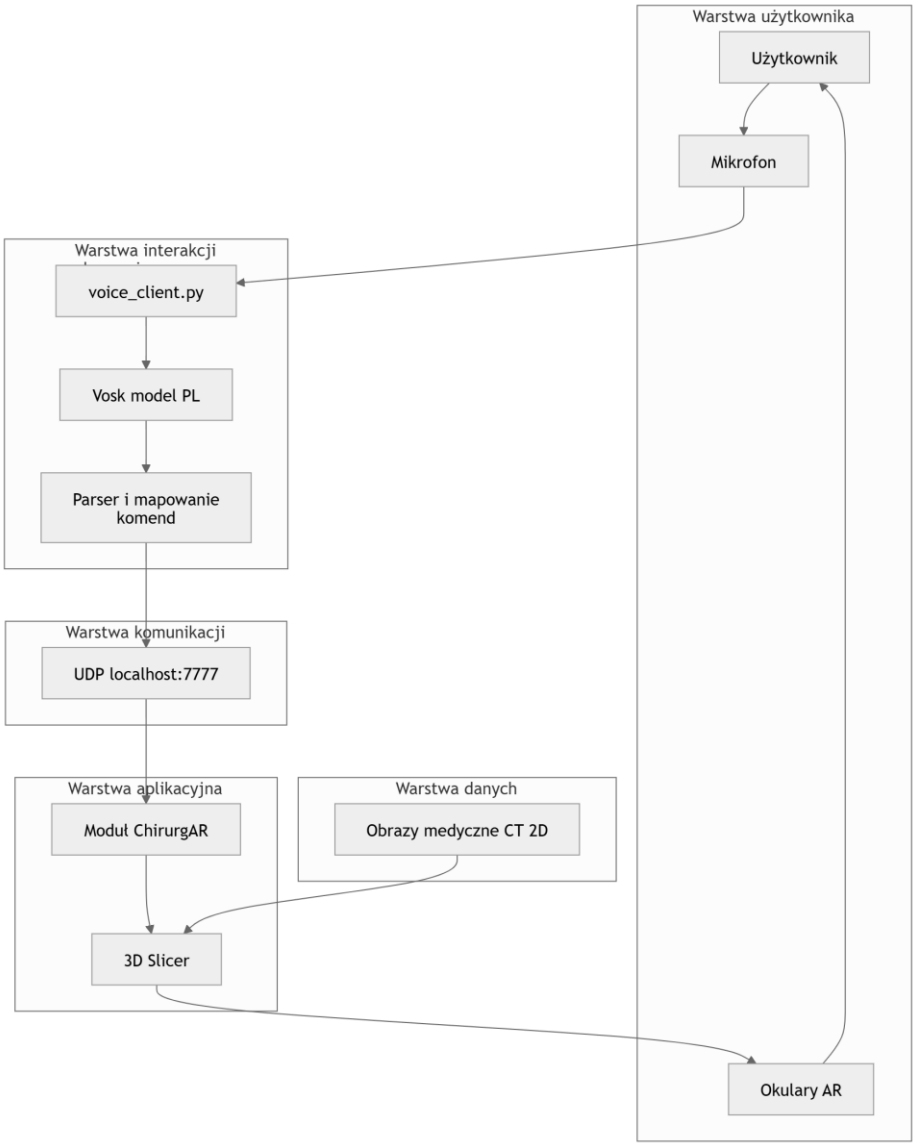
Celem projektu było opracowanie koncepcji oraz implementacji systemu wspomagającego pracę użytkownika (np. lekarza lub operatora danych medycznych). Całość projektu opierała się na zintegrowaniu technologii rozszerzonej rzeczywistości z bezdotykowym sterowaniem, przy użyciu komend głosowych. Celem systemu jest umożliwienie intuicyjnej i efektywnej interakcji obrazów 2D, pochodzących z tomografii komputerowej (CT), wyświetlanymi w środowisku rzeczywistym, przy użyciu okularów (*Augmented Reality*) AR. Kluczowym założeniem jest ograniczenie korzystania z tradycyjnych urządzeń wejściowych (np. myszka komputerowa), o ma szczególne znaczenie w środowisku medycznym, na blokach operacyjnych, w gabinetach zabiegowych. Projekt zakłada wykorzystanie okularów rozszerzonej rzeczywistości jako głównego interfejsu wizualnego, który pozwala na nakładanie obrazów medycznych na pole widzenia użytkownika bez utraty kontaktu z otoczeniem.

Zakres projektu obejmuje zaprojektowanie architektury systemu składającego się z modułu wizualizacji, modułu rozpoznawania mowy oraz warstwy komunikacyjnej umożliwiającej wymianę danych między komponentami. W ramach projektu realizowana jest implementacja podstawowych funkcji sterowania głosowego, takich jak nawigacja po warstwach obrazów tomograficznych, manipulacja widokiem (przybliżanie, przesuwanie), zmiana płaszczyzn oraz zarządzanie elementami wizualnymi. Projekt obejmuje również przygotowanie środowiska do wyświetlania danych w okularach AR, co może być zrealizowane poprzez integrację z urządzeniami wspierającymi rozszerzoną rzeczywistość lub symulację takiego środowiska. W zakres projektowe wchodzi także analiza ograniczeń technologicznych oraz przyszły rozwój opracowanej technologii.

3. Architektura systemu

Architektura systemu ma charakter rozproszony i opiera się na modelu klient–serwer, mimo że oba komponenty działają lokalnie na tej samej maszynie. Pierwszym elementem jest klient głosowy, zaimplementowany jako skrypt w języku Python (*voice_client.py*), który odpowiada za przechwytywanie sygnału audio z mikrofonu, jego przetwarzanie oraz interpretację w postaci tekstowej. Klient wykorzystuje model akustyczno-językowy

dostarczany przez Vosk, który operuje na strumieniu audio i generuje hipotezy tekstowe w czasie rzeczywistym. Proces ten obejmuje analizę fonetyczną, dekodowanie sekwencji dźwięków oraz dopasowanie do słownika językowego. Następnie rozpoznany tekst jest przetwarzany przez warstwę logiki aplikacyjnej, która mapuje konkretne frazy na zestaw predefiniowanych komend systemowych. Drugim komponentem jest moduł integracyjny działający wewnątrz środowiska 3D Slicer. Moduł ten jest dodawany poprzez mechanizm *„Additional module paths”* i ładowany podczas uruchamiania aplikacji. Po stronie Slicera implementowana jest warstwa odbioru komunikatów, która nasłuchuje na określonym porcie sieciowym (UDP, port 7777). Odbierane pakiety zawierają zakodowane komendy, które są następnie interpretowane i przekazywane do odpowiednich funkcji API Slicera. Operacje te obejmują manipulację widokami (*slice views*), kontrolę kamery, zmianę parametrów wizualizacji oraz operacje na obiektach typu *Markups Fiducials*, które reprezentują punkty anatomiczne w przestrzeni 3D. Całość architektury systemu zwizualizowano na Rysunku 1.



Rysunek 1. Schemat architektury systemu Chirurg AR – integracja okularów AR, sterowania głosowego oraz wizualizacji danych CT 2D.

4. Moduł klienta głosowego

Klient głosowy stanowi kluczowy element systemu. Odpowiada za realizację interakcji użytkownika z systemem za pomocą mowy. Jest to aplikacja uruchamiana niezależnie od środowiska 3D Slicer, zaimplementowana w języku Python (plik *voice_client.py*). Działa w trybie ciągłego nasłuchiwania i przechwytywania sygnału audio w czasie rzeczywistym. Głównym zadaniem klienta jest przechwytywanie dźwięku z mikrofonu, jego przetwarzanie przy użyciu biblioteki VOSK oraz konwersja wypowiedzi użytkownika na tekst, który następnie podlega dalszej analizie. Proces ten realizowany jest w oparciu o lokalny model językowy (*vosk-model-small-pl-0.22*), zawierający komponenty językowe i akustyczne dostosowane do języka polskiego. Dzięki temu możliwe jest rozpoznawanie mowy bez wykorzystania usług zewnętrznych, co eliminuje opóźnienia związane z transmisją danych oraz zwiększa kontrolę nad przetwarzaniem informacji.

Działanie klienta można podzielić na kilka etapów. W pierwszym z nich następuje akwizycja sygnału audio z urządzenia wejściowego, realizowana z wykorzystaniem strumienia wejściowego oraz mechanizmu *callback*, który przekazuje próbki dźwięku do kolejki przetwarzania. Strumień dźwięku jest następnie analizowany w sposób ciągły przez silnik rozpoznawania mowy Vosk, który przetwarza sygnał mowy i generuje odpowiadające mu hipotezy tekstowe. W implementacji wykorzystywane są wyłącznie wyniki końcowe (*final results*), co pozwala na zwiększenie stabilności działania systemu. Kolejnym etapem jest przetwarzanie rozpoznanego tekstu w warstwie logiki aplikacyjnej, obejmujące prostą analizę tekstu polegającą na tokenizacji (podziale wypowiedzi na słowa) oraz dopasowaniu słów kluczowych do zbioru predefiniowanych komend systemowych. Mechanizm ten opiera się na dopasowaniu wzorców (*pattern matching*), w którym określone frazy, takie jak „następna”, „poprzednia”, „przybliż” czy „reset”, są mapowane na konkretne akcje wykonywane w systemie.

Dodatkowo zaimplementowano obsługę liczebników słownych, co umożliwia interpretację parametrów liczbowych w poleceniach takich jak „o pięć” czy „o dziesięć”, przy czym system obsługuje również liczby zapisane w postaci cyfr. Uzyskane wartości wykorzystywane są następnie jako parametry sterujące wykonywanymi operacjami. System umożliwia także operacje na obiektach (tzw. *Landmarkach*) - w przypadku komend takich jak „pokaż” lub „ukryj”, nazwa obiektu jest wyodrębniana bezpośrednio z wypowiedzi użytkownika jako kolejny element polecenia, co pozwala na dynamiczne wskazywanie konkretnych struktur. Po zinterpretowaniu komendy następuje jej serializacja do formatu JSON oraz przygotowanie do transmisji. Klient wykorzystuje mechanizm komunikacji sieciowej oparty na protokole UDP do przesyłania danych do modułu wykonawczego działającego w środowisku 3D Slicer. Komunikaty wysyłane są na określony adres (najczęściej localhost) i port (7777), co umożliwia szybkie przekazywanie poleceń przy minimalnym narzucie komunikacyjnym. Z punktu widzenia implementacyjnego klient działa w pętli głównej, która cyklicznie pobiera dane z kolejki, przekazuje je do silnika rozpoznawania mowy, analizuje

uzyskany wynik i - w przypadku wykrycia poprawnej komendy - wysyła odpowiedni komunikat do odbiorcy.

5. Moduł Slicer 3D

Moduł ChirurgAR działający w środowisku 3D Slicer stanowi komponent wykonawczy systemu, odpowiedzialny za interpretację i realizację komend otrzymywanych z klienta głosowego. Jego główną rolą jest pośredniczenie pomiędzy warstwą interakcji (sterowanie głosowe) a warstwą wizualizacji danych medycznych, poprzez bezpośrednie wykorzystanie API Slicera do manipulacji sceną oraz widokami obrazów. Moduł został zrealizowany jako *Scripted Loadable Module* w Pythonie, zdefiniowany w pliku VoiceControl.py. W klasie głównej modułu ustawiono jego nazwę widoczną w interfejsie jako „ChirurgAR", przypisano go do kategorii „Chirurgia AR" oraz określono podstawowe informacje opisowe. Oznacza to, że moduł jest osadzony w standardowym mechanizmie rozszerzeń Slicera i nie wymaga ingerencji w kod źródłowy samej aplikacji. Z punktu widzenia struktury projektu moduł jest budowany jako skryptowy moduł Pythona z wykorzystaniem pliku CMakeLists.txt, w którym zdefiniowano nazwę modułu jako VoiceControl, wskazano główny skrypt Pythona oraz zasoby interfejsu, takie jak ikona i plik *.ui*. Warstwa interfejsu użytkownika modułu została zrealizowana w klasie *VoiceControlWidget*. W metodzie setup() tworzony jest prosty panel sterowania zawierający dwa przyciski oraz etykietę informacyjną. Pierwszy przycisk służy do uruchomienia nasłuchiwania komunikatów UDP na porcie 7777, natomiast drugi do zatrzymania nasłuchiwania. Dodatkowo etykieta informacyjna wyświetla aktualny stan modułu, np. oczekiwanie na start, poprawne uruchomienie nasłuchu lub zatrzymanie działania. W praktyce oznacza to, że moduł posiada własny, minimalistyczny interfejs sterujący bezpośrednio w oknie Slicera i nie działa całkowicie w tle. To rozwiązanie ma również charakter demonstracyjny, użytkownik widzi stan połączenia oraz może ręcznie aktywować lub dezaktywować komunikację. Dalsza część działania modułu opiera się na mechanizmie komunikacji sieciowej wykorzystującym klasę *QUdpSocket*, która odpowiada za odbieranie pakietów danych z klienta głosowego. Po uruchomieniu nasłuchiwania moduł wiąże się z portem 7777 i oczekuje na przychodzące komunikaty. W momencie ich odebrania wywoływana jest funkcja obsługi, w której dane są dekodowane i interpretowane jako struktura JSON. Z komunikatu wyodrębniane są podstawowe parametry sterujące, takie jak płaszczyzna (*plane*), akcja (*action*), wartość (*value*) oraz opcjonalna etykieta (*label*). Kluczowa logika systemu została zaimplementowana w metodzie wykonawczej, która mapuje otrzymane komendy na konkretne operacje dostępne w środowisku 3D Slicer. W zależności od typu polecenia moduł umożliwia nawigację pomiędzy warstwami obrazów (zmiana offsetu przekrojów), manipulację widokiem (przesuwanie oraz przybliżanie), a także sterowanie układem interfejsu (np. skupienie na jednej płaszczyźnie lub reset widoku). Dodatkowo zaimplementowano obsługę punktów anatomicznych (*landmarków*), gdzie na podstawie przekazanej etykiety możliwe jest ich dynamiczne pokazywanie lub ukrywanie. Operacja ta realizowana jest poprzez przeszukiwanie obiektów sceny i zmianę ich widoczności. Warto podkreślić, że wszystkie operacje wykonywane są bezpośrednio na strukturze MRML (*Medical Reality Markup*

Language), co oznacza, że moduł ingeruje bezpośrednio w aktualny stan sceny i danych medycznych.

6. Komunikacja i protokół wymiany danych

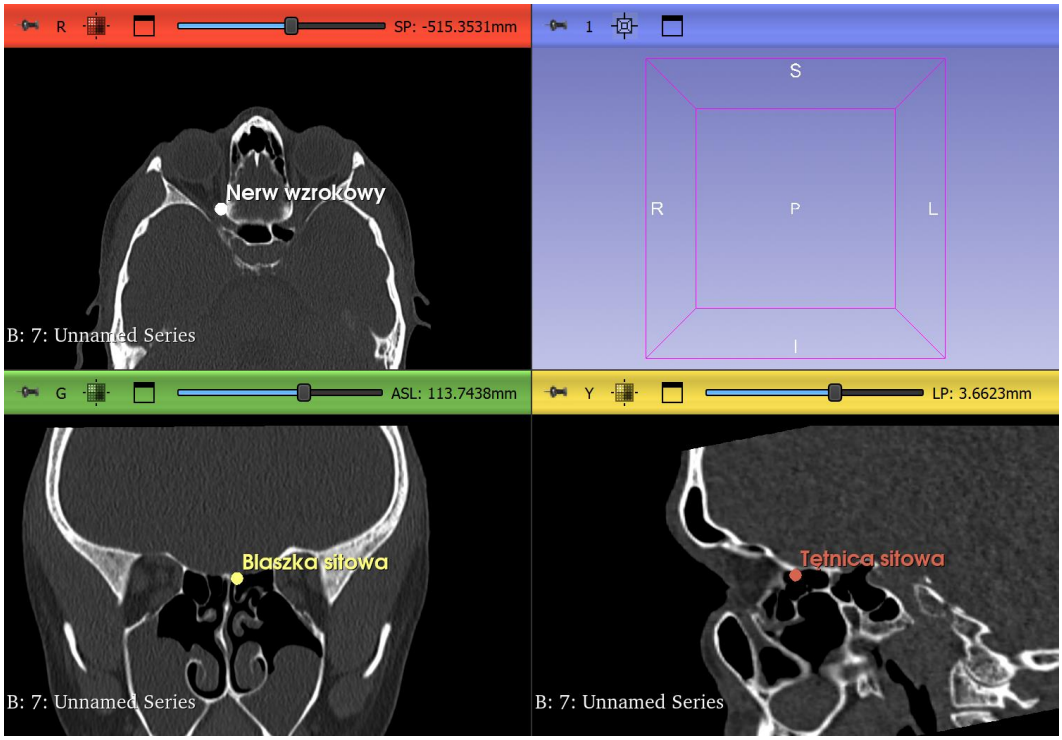
W celu zapewnienia separacji procesów oraz minimalizacji latencji sterowania, w systemie zaimplementowano warstwę komunikacyjną opartą na architekturze klient – serwer. Wybór protokołu UDP podyktowany został specyfiką pracy śródoperacyjnej – priorytetem jest natychmiastowa reakcja interfejsu przy minimalnym narzucie stosu sieciowego.

Wymiana informacji odbywa się poprzez serializowane obiekty tekstowe w standardzie JSON. Każdy datagram zawiera ustrukturyzowany ładunek użyteczny (*payload*), który definiuje stan docelowy rzutni w programie 3D Slicer:

Klucz	**Typ**	**Opis parametrów**
plane	String	Identyfikator płaszczyzny: axial, coronal, sagittal lub global.
action	String	Identyfikator funkcji API (np. next_slice, zoom_in, show_landmark).
value	Integer	Modyfikator intensywności (np. skok warstwy o *n* jednostek).
label	String	(Opcjonalnie) Etykieta struktury anatomicznej w węźle *Markups*.

7. Funkcjonalność systemu

Warstwa funkcjonalna systemu obejmuje zestaw komend głosowych umożliwiających sterowanie środowiskiem wizualizacji medycznej. Podstawową grupę funkcji stanowi nawigacja pomiędzy warstwami obrazów medycznych. Użytkownik może przemieszczać się pomiędzy kolejnymi przekrojami w płaszczyznach osiowej, strzałkowej i czołowej, co realizowane jest poprzez zmianę offsetu wyświetlanej warstwy. Dodatkowo system umożliwia wykonywanie operacji z określoną intensywnością dzięki zastosowaniu parametru liczbowego (np. przesunięcie o kilka warstw jednocześnie). System obsługuje również funkcje manipulacji widokiem, takie jak przybliżanie, oddalanie oraz przesuwanie obrazu w określonych kierunkach. Komenda „reset” przywraca domyślny układ widoków, natomiast polecenie „zostaw” umożliwia maksymalizację wybranego okna i skupienie się na jednej płaszczyźnie. Dodatkowo system wspiera zarządzanie punktami anatomicznymi (*Markups Fiducials*), umożliwiając ich wyświetlanie i ukrywanie na podstawie nazw. Mechanizm ten opiera się na identyfikacji obiektów w scenie oraz zmianie ich właściwości widoczności. Obecnie oznaczono trzy struktury anatomiczne: tętnice sitową, nerw wzrokowy oraz blaszkę sitową. Jak wykazała literatura są to struktury najbardziej narażone podczas operacji. Istotnym elementem jest także implementacja prostego parsera liczebników w języku naturalnym, który pozwala na interpretację wartości takich jak „pięć”, „dziesięć” czy „dwadzieścia”. Szczegółowy wykaz dopuszczalnych fraz sterujących oraz ich mapowanie na format wymiany danych JSON został przedstawiony w rozdziale 8: Specyfikacja interfejsu głosowego.



Rysunek 2. Przykład anotacji landmarków (tętnica sitowa, nerw wzrokowy, blaszka sitowa).

8. Specyfikacja interfejsu głosowego

Niniejszy rozdział stanowi definicję dopuszczalnych fraz sterujących oraz ich bezpośrednie przełożenie na logikę medyczną systemu.

8.1. Klasyfikacja komend nawigacyjnych (Widoki 2D)

- **Nawigacja warstwowa:** [płaszczyzna] + następna/kolejna/poprzednia/cofnij + [o wartość]
  - *Skutek:* Inkrementacja lub dekrementacja parametru SliceOffset.
- **Skalowanie widoku:** [płaszczyzna] + przybliż/oddal
  - *Skutek:* Proporcjonalna zmiana wartości *Field of View* (FOV).
- **Przesuwanie obrazu (Pan):** [płaszczyzna] + lewo/prawo/góra/dół
  - *Skutek:* Translacja macierzy widoku w osiach rzutni.
- **Zarządzanie układem:** * **reset**: Przywrócenie domyślnej matrycy widoków (układ 4-okienkowy).
  - **zostaw/zostać**: Maksymalizacja wybranej płaszczyzny (*Single View Mode*).

8.2. Zarządzanie strukturami anatomicznymi (*Landmarks*)

Komendy dedykowane do zarządzania widocznością kluczowych struktur oznaczonych w scenie MRML:

- **Aktywacja widoczności:** pokaż + [etykieta_punktu]

- **Dezaktywacja widoczności:** ukryj/schowaj + [etykieta_punktu]

Przykład interpretacji złożonej

Frazie: *„czołowa następna o dziesięć”* odpowiada komunikat:
{"plane": "coronal", "action": "next_slice", "value": 10}

9. Instalacja i konfiguracja

Proces instalacji systemu wymaga ręcznej konfiguracji dwóch niezależnych środowisk: klienta głosowego oraz modułu 3D Slicer.

9.1. Przygotowanie Klienta Głosowego

Wymagane jest pobranie modelu akustycznego *vosk-model-small-pl-0.22* i umieszczenie go bezpośrednio w katalogu aplikacji klienckiej (*voice_client)*. Należy upewnić się, że zainstalowano wymagane biblioteki zdefiniowane w pliku *requirements.txt*.

9.2. Integracja z oprogramowaniem 3D Slicer

1. W menu 3D Slicer przejdź do: **Edit -> Application Settings -> Modules**.
2. W sekcji **Additional module paths** kliknij **Add** i wskaż folder *slicer_module*.
3. Zrestartuj aplikację 3D Slicer w celu poprawnej indeksacji nowych skryptów Python.
4. Wyszukaj moduł **ChirurgAR** i aktywuj przycisk **„URUCHOM NASŁUCH (Port 7777)”**.

9.3. Procedura startowa systemu

System wymaga równoległego działania obu komponentów. Skrypt voice_client.py musi zostać zainicjowany jako niezależny proces z poziomu terminala systemowego. Po nawiązaniu nasłuchu przez Slicer, komunikacja UDP odbywa się w czasie rzeczywistym.

9.4. Zakończenie pracy systemu

W celu zakończenia pracy systemu należy wywołać komendę “Zakończ”. Spowoduje to zakończenie pracy modelu Vosk oraz 3D Slicer.

10. Testy funkcjonalne

Opisane powyżej rozwiązanie zostało przetestowane podczas zajęć laboratoryjnych. W tym celu wykorzystano gogle EPSON Moverio BT-45 CS. System został przetestowany pod względem precyzji wychwytywania komend. W tym przypadku system działał w sposób poprawny. W przypadku głosu męskiego, niższego system bez problemu wychwytywał kolejne komendy. Nieco gorzej radził sobie z głosem damskim, który jest wyższy. Podczas testów zauważono, że lepszym rozwiązaniem byłoby zaimplementować rozwiązani umożliwiające maksymalizację okna przeglądu w 3D Slicer. Spowodowałoby to znaczne zwiększenie efektywności całości projektu. Testy funkcjonalne przebiegły pomyślnie, potwierdzając praktyczne zastosowanie zaimplementowanego rozwiązania.

11. Ograniczenia systemu

Architektura systemu Chirurg AR w obecnej fazie prototypu opiera się na świadomych kompromisach technologicznych. Kluczowym ograniczeniem jest jednokierunkowa komunikacja wykorzystująca bezpołączeniowy protokół UDP. Choć minimalizuje on opóźnienia w transmisji lokalnej, warstwa sieciowa nie posiada wbudowanych mechanizmów potwierdzania odbioru ani kontroli stanu połączenia. System nie realizuje zamkniętego sprzężenia zwrotnego – po wysłaniu polecenia klient głosowy nie otrzymuje od środowiska 3D Slicer potwierdzenia wykonania operacji. Wymusza to na użytkowniku ciągłą weryfikację wzrokową i stwarza ryzyko nieświadomości awarii, co dyskwalifikuje obecną wersję z bezpośredniego użycia klinicznego. Brakuje także mechanizmów obsługi błędów, takich jak utrata połączenia, błędne komendy czy problemy z dostępem do urządzeń wejściowych.

Kolejne ograniczenie wynika z wymogu przetwarzania danych medycznych w trybie offline. Wymusiło to zastosowanie lokalnego modelu akustycznego Vosk, co pozbawiło system chmurowej, zaawansowanej analizy semantycznej. Ograniczenie to zniwelowano poprzez implementację w logice klienta sztywnych słowników, zawierających zamkniętą pulę predefiniowanych odmian fleksyjnych słów kluczowych. W efekcie system interpretuje intencje wyłącznie na podstawie deterministycznego dopasowywaniu wzorców, ignorując szerszy kontekst wypowiedzi. Dodatkowo brak dedykowanej warstwy odszumiania sygnału obniża niezawodność algorytmu w hałaśliwym środowisku bloku operacyjnego.

Ostatnim elementem jest ograniczenie, które zaobserwowano w fazie testów. Jest to zwrócenie uwagi na potrzebę maksymalizacji okna przeglądu obrazów CT. Obecne rozwiązanie zawiera całe okno aplikacyjne, co powoduje zmniejszenie obrazów kluczowych oraz zajmowanie miejsca w polu widzenia, przez niepraktyczne elementy obrazu 3D Slicer. Ograniczenie to można rozwiązać stosując chociażby narzędzie wycinania obrazu i dopiero wtedy udostępnianie obrazu na ekranie gogli.

12. Podsumowanie

Projekt ChirurgAR–Voice Control stanowi funkcjonalny system umożliwiający sterowanie środowiskiem wizualizacji medycznej 3D Slicer przy użyciu komend głosowych przetwarzanych lokalnie za pomocą technologii Vosk. Rozwiązanie to wpisuje się w aktualny kierunek rozwoju interfejsów użytkownika w medycynie, gdzie dąży się do minimalizacji kontaktu fizycznego z urządzeniami wejściowymi oraz poprawy ergonomii pracy, szczególnie w środowiskach wymagających zachowania sterylności. Z technicznego punktu widzenia projekt poprawnie realizuje podstawowe założenia architektoniczne systemu rozproszonego opartego na komunikacji klient–serwer. Wyraźny podział na komponent odpowiedzialny za rozpoznawanie mowy oraz moduł wykonawczy w środowisku 3D Slicer jest rozwiązaniem przejrzystym i zgodnym z dobrymi praktykami projektowania systemów. Zastosowanie lokalnego modelu językowego pozwala na niezależność od usług zewnętrznych, co w kontekście przetwarzania danych medycznych stanowi istotną zaletę. Dodatkowo wykorzystanie komunikacji UDP umożliwia osiągnięcie niskich opóźnień, co jest kluczowe w interakcjach czasu rzeczywistego, choć jednocześnie wiąże się z określonymi kompromisami w zakresie niezawodności transmisji. Warstwa funkcjonalna systemu została zaprojektowana w sposób umożliwiający realizację podstawowych operacji na danych obrazowych, takich jak nawigacja po warstwach, manipulacja widokiem czy zarządzanie punktami anatomicznymi. Implementacja prostego mechanizmu interpretacji języka naturalnego, w tym liczebników, zwiększa intuicyjność obsługi i stanowi istotny krok w kierunku bardziej zaawansowanych interfejsów głosowych. Jednocześnie zakres funkcjonalny systemu pozostaje ograniczony do z góry zdefiniowanego zestawu komend, co wskazuje na brak zaawansowanego przetwarzania języka naturalnego oraz brak adaptacyjności systemu do różnych użytkowników i kontekstów pracy.

Pomimo tych pozytywnych aspektów, projekt posiada pewne ograniczenia, które uniemożliwiają jego wykorzystanie w środowisku produkcyjnym lub klinicznym. Krytycznym obszarem jest bezpieczeństwo. W obecnej formie system nie implementuje żadnych mechanizmów szyfrowania transmisji ani uwierzytelniania użytkownika czy komponentów systemu. W kontekście potencjalnego zastosowania w medycynie, gdzie przetwarzane są dane wrażliwe, stanowi to poważne uchybienie. Dodatkowo wykorzystanie protokołu UDP bez mechanizmów kontroli integralności i dostarczenia danych oznacza brak gwarancji wykonania komend, co w środowisku klinicznym jest niedopuszczalne. Nie przeprowadzono testów z udziałem użytkowników końcowych ani analizy użyteczności (UX), co uniemożliwia ocenę rzeczywistej wartości systemu w praktyce. Istotnym ograniczeniem jest także brak integracji z rzeczywistym ekosystemem medycznym.

W celu zwiększenia efektywności, niezawodności oraz potencjalnej użyteczności systemu ChirurgAR w przyszłości, można rozważyć szereg kierunków rozwoju. Przede wszystkim istotne jest rozszerzenie warstwy bezpieczeństwa poprzez implementację szyfrowania komunikacji (np. TLS) oraz mechanizmów uwierzytelniania użytkownika i autoryzacji dostępu. W kontekście medycznym konieczne byłoby również zapewnienie zgodności z regulacjami dotyczącymi ochrony danych, takimi jak RODO czy standardy systemów informatycznych w ochronie zdrowia. Kolejnym krokiem powinno być zwiększenie niezawodności komunikacji poprzez zastąpienie lub rozszerzenie protokołu UDP o mechanizmy potwierdzania odbioru, kolejkowania komunikatów oraz obsługi błędów. Możliwe jest także zastosowanie protokołów bardziej odpornych na utratę danych, takich jak TCP lub rozwiązania hybrydowe. W dalszym rozwoju projektu niezbędne jest także

zoptymalizowanie algorytmów odszumiających sygnał wejściowy, aby sprostać trudnym warunkom akustycznym pracującej aparatury na bloku operacyjnym.

Podsumowując, ChirurgAR – Voice Control należy uznać za dopracowany prototyp technologiczny, który w przekonujący sposób udowadnia wykonalność integracji rozpoznawania mowy ze środowiskiem 3D Slicer. Projekt potwierdza ogromny potencjał interfejsów bezdotykowych w pracy z danymi medycznymi i stanowi solidną, ugruntowaną bazę architektoniczną do dalszego rozwoju. Warunkiem koniecznym do przekształcenia prototypu w pełnoprawne narzędzie kliniczne jest jednak domknięcie kwestii bezpieczeństwa, niezawodności sieciowej oraz docelowej wizualizacji.